

Machine Learning II

Generalization, Model Evaluation & Selection, Hyperparameter Optimization, and Model Ensembles

Chengwei QIN

AI Thrust, Information Hub

2025.10.30

- 1 Generalization: The Bias-Variance Trade-off
- 2 Model Evaluation & Selection
- 3 Hyperparameter Optimization
- 4 Model Ensembles

Generalization: The Bias-Variance Trade-off

The Goal of Machine Learning

Question: What is our *actual* goal?

- Is it to get 100% accuracy on our training data?
- **No.** That's easy to “cheat” by memorizing the training set.
- **Analogy:** A student who memorizes 10 practice questions but doesn't grasp the underlying concepts will fail the real exam.

The True Goal of Generalization

Build a model that performs well on **new, unseen data** drawn from the **same distribution** as the training data.

Generalization is the essence of learning — the ability to apply knowledge beyond the examples seen.

The Problem: Risk Minimization

How do we formalize "performing well"?

- We have a **loss function** $\ell(f(x), y)$ that measures how bad a prediction $f(x)$ is compared to the true label y .
- We want to find a function f that minimizes the **True Risk** (or Expected Risk), $\mathcal{R}(f)$:

$$\mathcal{R}(f) = \mathbb{E}_{(x,y) \sim \mathcal{P}}[\ell(f(x), y)]$$

- This is the *true average loss* over all possible data points.

The Catch:

- We **do not know** the true data distribution $\mathcal{P}$.
- All we have is a finite training set $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^N$.

The Strategy: Empirical Risk Minimization (ERM)

- We minimize the **Empirical Risk** (the average loss on our training set) instead:

$$\mathcal{R}_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i), y_i)$$

The Core Tension: Underfitting vs. Overfitting

Minimizing empirical risk (training error) is a good proxy, but it leads to a fundamental tension:

Underfitting

- **Problem:** The model is too simple to capture the underlying patterns in the data.
- **Symptom:** High training error *and* high test error.
- **Analogy:** The student who didn't study at all. Fails practice and real exams.

Overfitting

- **Problem:** The model is too complex. It memorizes the training data, including its noise.
- **Symptom:** Very low training error but *high* test error.
- **Analogy:** The student who memorized the 10 practice questions. Aces the practice, fails the real exam.

Generalization is the "sweet spot" in between.

A Framework for Generalization: Bias & Variance

We can formally decompose the expected test error of a model.

- **Bias:**

- The error from the model's "stubbornness" or incorrect assumptions.
- How far off is the *average* model (trained on all possible datasets) from the true function?
- **High Bias** → **Underfitting**. (e.g., using a linear model for a sine wave)

- **Variance:**

- The error from the model's "sensitivity" to the specific training data it saw.
- How much do models (trained on *different* datasets) vary from each other?
- **High Variance** → **Overfitting**. (e.g., a high-degree polynomial that wiggles to catch every point)

Bias-Variance Decomposition (The Math)

For squared loss, we can decompose the expected test error at a point x :

$$\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - y)^2]$$

Where $\hat{f}_{\mathcal{D}}$ is the model trained on a specific dataset $\mathcal{D}$.

This decomposes into three parts:

$$\text{Error}(x) = \underbrace{(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - f^*(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{V}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible Noise}}$$

- **Bias²**: How far the *average prediction* is from the *true function* $f^*(x)$.
- **Variance**: The variance of model predictions around their average.
- **Noise(σ^2)**: The unavoidable error inherent in the data itself. We can't reduce this.

The Bias-Variance Trade-off

There is a fundamental trade-off between bias and variance, controlled by **model complexity**.

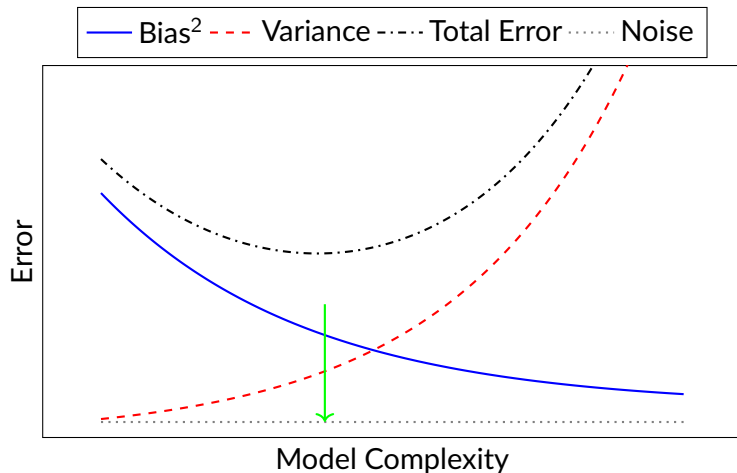
- **Low Complexity Model** (e.g., Linear Regression)
 - Low Variance (all models look similar)
 - High Bias (likely wrong assumption)
- **High Complexity Model** (e.g., 15-degree Polynomial, deep NN)
 - High Variance (models are very sensitive to data)
 - Low Bias (can fit almost anything)

The Implementation of Generalization

Find the "sweet spot" of complexity that minimizes the **Total Error** ($\text{Bias}^2 + \text{Variance}$).

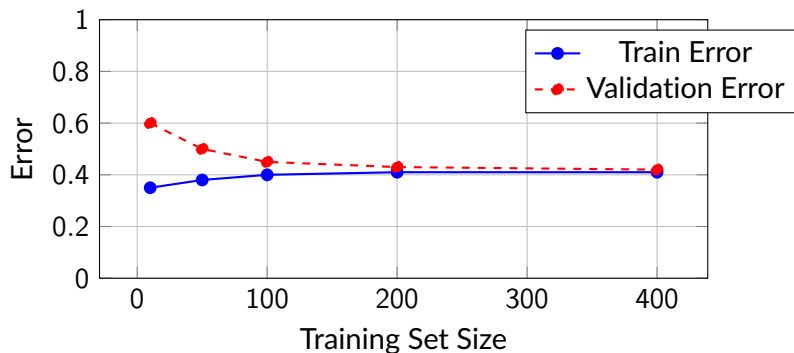
Illustration: Bias vs. Variance Curves

This plot shows the classic relationship:



Diagnosing High Bias (Underfitting)

Symptom: Both training and validation error are high and converged.



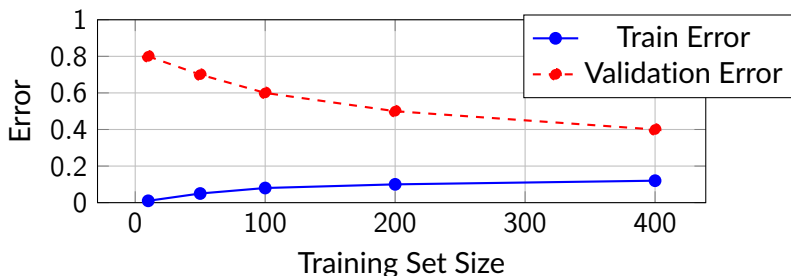
Diagnosis: The model is too simple.

Remedy:

- Use a more complex model (e.g., more features, higher degree polynomial, bigger NN).
- **Getting more data will NOT help!**

Diagnosing High Variance (Overfitting)

Symptom: Large gap between low training error and high val error.



Diagnosis: The model is too complex and is memorizing noise.

Remedy:

- **Get more data!** (The gap will shrink).
- Use a simpler model.
- Add/increase regularization (e.g., lower C , add dropout).

Practical Demo: Polynomial Under/Overfitting

Goal: Show how model complexity affects bias and variance.

- **True Function:** $y = \text{sinc}(x)$ on $[-5, 5]$.
- **Data:** Noisy samples from the true function (Gaussian noise).
- **Models:** Polynomial regression with degrees 1, 4, and 15 (linear regression on polynomial features).
- **Evaluation:** Mean Squared Error (MSE) on a dense test grid; visual comparison of fits.

What to Look For

Degree 1 $\Rightarrow$ **high bias (underfitting)**; Degree 15 $\Rightarrow$ **high variance (overfitting)**; Degree 4 $\Rightarrow$ **balanced fit**.

All implementation details and plots are provided in the accompanying Jupyter notebook.

Practical Demo: Results & Insights

Visual Summary: (See notebook for plots comparing Degree 1, 4, 15.)

- **Degree 1 – Underfitting (High Bias):** Misses curvature; both training and test errors remain high.
- **Degree 4 – Good Fit (Balanced):** Captures the main trend; lowest test MSE among the three.
- **Degree 15 – Overfitting (High Variance):** Oscillatory fit that chases noise; very low training error but high test error.

Key Takeaways

- Increasing model complexity *reduces bias* but *increases variance*.
- The optimal complexity is the one that minimizes **test** error, not training error.
- Use diagnostic tools (learning/validation curves) and a proper hold-out protocol to pick complexity.

Model Evaluation & Selection

Why Evaluation & Selection?

Motivation

Generalization is our end goal. Evaluation *measures* it; selection *improves* it. Without a sound protocol, we either underfit (high bias) or overfit (high variance), and reported numbers become misleading.

What can go wrong?

- **Leakage:** Using information from validation/test in training or tuning.
- **Overfitting to Validation:** Re-using the same val split until it becomes a de facto train set.
- **Unrepresentative Split:** Single random split yields brittle conclusions.

What do we want?

- **Unbiased** estimate of true risk (hold-out test).
- **Low-variance** selection signal (cross-validation).
- **Alignment** with bias-variance trade-off from Section 1.

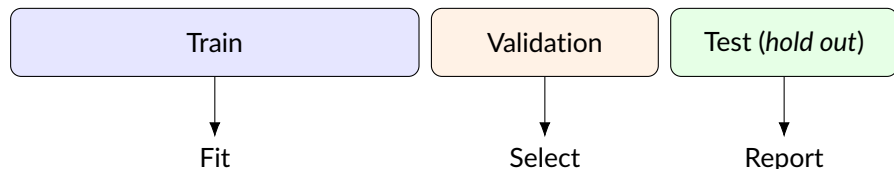
From Bias–Variance to Practice

- **Low complexity** $\Rightarrow$ high bias: evaluation should reveal *converged* high train/val errors (diagnose underfitting).
- **High complexity** $\Rightarrow$ high variance: evaluation should reveal a *gap* between train and val errors (diagnose overfitting).
- **Selection** searches for the complexity/regularization that minimizes estimated test error.

Design Principle

Separate *what the algorithm learns* (training) from *what the practitioner decides* (selection), and reserve a **final untouched** test set for reporting.

Clean Data-Splitting Protocol



- **Train:** Fit model parameters.
- **Validation:** Choose model type/HPs/early stopping.
- **Test (once):** Final, unbiased estimate.

Example split: 60% Train / 20% Val / 20% Test (for large N).

Golden Rule

Never use test data for training or selection. Use it once at the very end.

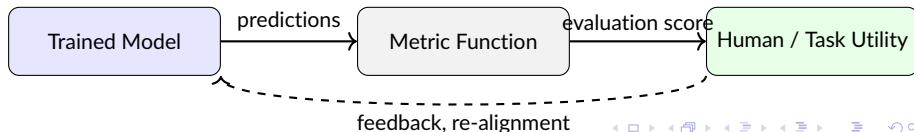
Model Evaluation: Why Metrics Matter

Core Idea

Metrics are the bridge between **model performance** and **human or task preference**. They quantify how well the model aligns with what we truly care about in the downstream task.

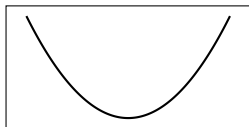
Purpose of Evaluation

- Provide an *objective measurement* of model behavior using quantitative criteria.
- Translate **human goals** (accuracy, safety, efficiency, fairness, satisfaction) into measurable signals.
- Serve as a **feedback signal** for improvement, selection, and deployment decisions.



Regression: MSE & RMSE — Preference Alignment

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}$$



Interpretations: Quadratic *disutility* for residuals: a proxy for human preference against large errors.

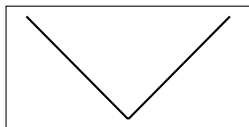
Pros/Cons

- *Pros:* Smooth, differentiable; emphasizes impactful errors.
- *Cons:* Outlier-sensitive; scale-dependent.

```
1 from sklearn.metrics import mean_squared_error
2 mse = mean_squared_error(y_true, y_pred, squared=True)
3 rmse = mean_squared_error(y_true, y_pred, squared=False)
```

Regression: MAE – Preference Alignment

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$



Interpretations: Linear disutility for residuals: aligns with preference for fairness and robustness to outliers.

Pros/Cons

- *Pros:* Robust to outliers; interpretable in original units; median-optimal.
- *Cons:* Non-smooth at 0; less sensitive to large errors.

```
from sklearn.metrics import mean_absolute_error  
mae = mean_absolute_error(y_true, y_pred)
```

Classification: Precision – Preference Alignment

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$



Predicted Positive

Interpretations: Precision measures reliability of positive predictions – preferred when false alarms are costly.

Pros/Cons

- *Pros:* Direct proxy for alert reliability; useful when attention is scarce.
- *Cons:* Ignores missed positives (recall).

```
1 from sklearn.metrics import precision_score
2 prec = precision_score(y_true, y_pred_labels, average='
    binary')
```

Classification: Recall – Preference Alignment

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

TP	FN
----	----

Actual Positive

Interpretations: Recall measures coverage of actual positives – preferred when missing positives is costly.

Pros/Cons

- *Pros:* Direct proxy for miss risk; important for safety/health applications.
- *Cons:* Ignores false positives (precision).

```
1 from sklearn.metrics import recall_score
2 rec = recall_score(y_true, y_pred_labels, average='
    binary')
```

Classification: F1 – Preference Alignment

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

TP	FP	FN
----	----	----

Interpretations: F1 balances precision and recall – preferred when false positives and false negatives are equally important.

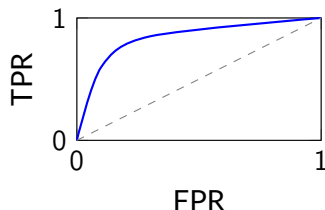
Pros/Cons

- *Pros:* Single number for PR trade-off; robust to class imbalance.
- *Cons:* Ignores true negatives; hides threshold choice.

```
1 from sklearn.metrics import f1_score
2 f1 = f1_score(y_true, y_pred_labels, average='binary')
```

Classification: ROC-AUC – Preference Alignment

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$



Interpretations: ROC-AUC measures ranking quality – preferred when you want good ordering for later threshold tuning.

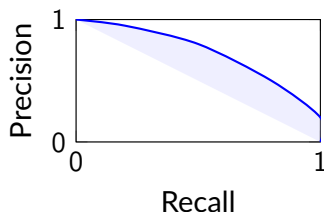
Pros/Cons

- *Pros:* Threshold-free; less sensitive to class imbalance.
- *Cons:* May hide poor performance on positive class under heavy imbalance.

```
1 from sklearn.metrics import roc_auc_score, roc_curve
2 auc = roc_auc_score(y_true, y_scores)
3 fpr, tpr, thr = roc_curve(y_true, y_scores)
```

Classification: PR-AP — Preference Alignment

$$AP = \int_0^1 \text{Precision}(r) dr$$



Interpretations: PR-AP summarizes precision-recall trade-off — preferred when positive class is rare and missing positives is costly.

Pros/Cons

- *Pros:* Reflects positive-class utility; useful for imbalanced tasks.
- *Cons:* Sensitive to prevalence; threshold still needed for deployment.

```
1 from sklearn.metrics import average_precision_score,  
  precision_recall_curve  
2 ap = average_precision_score(y_true, y_scores)  
3 prec, rec, thr = precision_recall_curve(y_true, y_scores)
```

- **Regression**

- Large-error risk $\Rightarrow$ MSE/RMSE
- Robustness $\Rightarrow$ MAE

- **Classification**

- Few positives $\Rightarrow$ PR-AP
- Balanced classes $\Rightarrow$ ROC-AUC
- Operating point needed $\Rightarrow$ Precision/Recall/F1

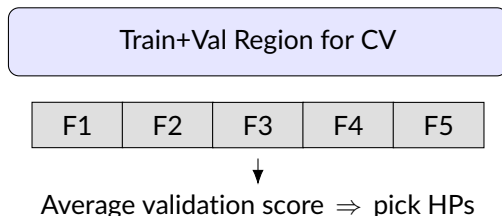
- **Always**

- Report CIs or CV mean $\pm$ std
- Disclose threshold

Model Selection: Core Workflow

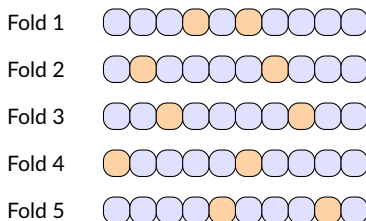
Goal: Pick model/HPs expected to generalize best.

- Define search space (model types, HP ranges).
- Use **K-Fold CV** on Train+Val portion to estimate performance per setting.
- Choose the setting with best *average* CV score.
- Retrain the selected configuration on Train+Val; **evaluate once** on Test.



Stratified K-Fold (Classification)

Preserve class proportions per fold; reduces variance of selection signal.

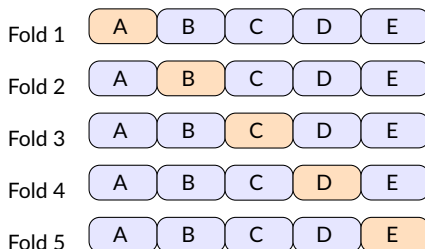


 Positive Class (minority)  Negative Class (majority)

```
1 from sklearn.model_selection import StratifiedKFold,
   cross_val_score
2 skf = StratifiedKFold(n_splits=5, shuffle=True,
   random_state=42)
3 scores = cross_val_score(model, X, y, cv=skf, scoring='
   f1')
```

Group K-Fold

Keep all samples from the same entity (patient/user) in the same fold; avoids optimistic bias.

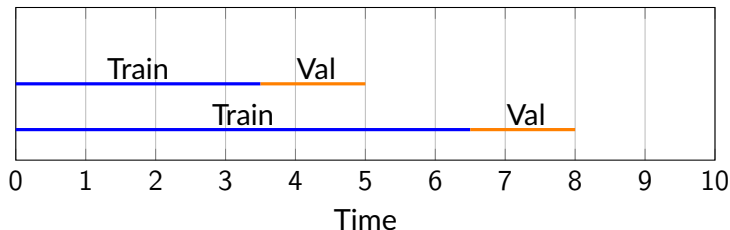


 Validation Groups  Training Groups

```
1 from sklearn.model_selection import GroupKFold,
   cross_val_score
2 # groups: array-like indicating entity id per sample
3 gkf = GroupKFold(n_splits=5)
4 scores = cross_val_score(model, X, y, groups=groups, cv=
   gkf, scoring='roc_auc')
```

Time Series Split

Train on the past, validate on the immediate future; use rolling or expanding window; never shuffle.

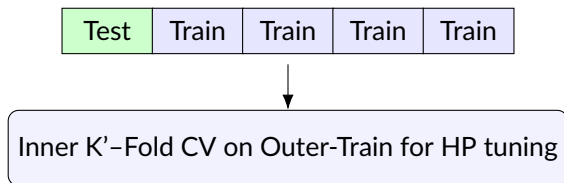


```
1 from sklearn.model_selection import TimeSeriesSplit,
   cross_val_score
2 tscv = TimeSeriesSplit(n_splits=5)
3 scores = cross_val_score(model, X, y,
4                           cv=tscv, scoring='
5                           neg_mean_squared_error')
6 rmse = (-scores.mean())**0.5
7 print(rmse)
```

Nested Cross-Validation (for Unbiased Reporting)

Problem: Using the same CV for both selection and reporting is optimistically biased. **Nested CV:**

- **Outer loop** (K folds): evaluation folds.
- **Inner loop** (K'): selection & tuning within each outer-train split.
- Report mean $\pm$ std of outer test scores.



Hyperparameter Optimization

Why Hyperparameter Optimization?

Motivation

Models expose *hyperparameters* (HPs) that control capacity, regularization, and training dynamics. Choosing them well is crucial to reach the generalization “sweet spot.”

Why it matters

- **Accuracy Robustness:** HPs alter bias–variance; poor choices lead to underoverfitting.
- **Efficiency:** Training time, memory, and inference cost depend on HPs.
- **Reliability:** Stable, reproducible performance requires principled tuning protocols.
- **Fair Comparison:** Methods must be compared under *comparable* tuning budgets.

What Are Hyperparameters?

Definition: Configuration variables set *outside* empirical risk minimization (e.g., λ in Ridge, tree depth, learning rate, data augmentation strength).

Types:

- **Continuous:** learning rate, weight decay.
- **Discrete:** number of estimators, layers, neighbors.
- **Categorical:** optimizer choice, kernel type, activation.
- **Conditional:** HPs active only when another choice is made (e.g., if kernel=rbf then tune γ).

Design Principle

Well-posed search spaces (ranges, scales, conditions) matter as much as the search algorithm.

Bi-level view

$$\theta^*(\lambda) = \arg \min_{\theta} \hat{\mathcal{R}}_{\text{train}}(\theta; \lambda), \quad \lambda^* = \arg \min_{\lambda \in \Lambda} \hat{\mathcal{R}}_{\text{val}}(\theta^*(\lambda))$$

- Inner level trains model parameters θ given HPs λ .
- Outer level selects HPs to minimize generalization proxy (val risk or outer-CV).
- Objective is **noisy**, **expensive**, possibly **non-smooth** **non-convex**.

Implication

Choice of **estimator** (validation vs. outer-CV) and **budget** (trials, epochs, data fraction) defines the HPO landscape.

Evaluation Protocols for HPO

Clean separation

- Use **Train** to fit parameters for each λ .
- Use **Validation** (or inner CV) to select λ .
- Report **once** on Test (or outer CV) after selection.

When data are scarce

- **K-Fold CV** in the outer loop for selection signal with lower variance.
- **Nested CV** to obtain an *unbiased* generalization estimate after hyperparameter optimization.

Reporting Norm

Disclose search space, algorithm, budget, CV protocol, seed(s), and the final test estimate with CI or mean $\pm$ std.

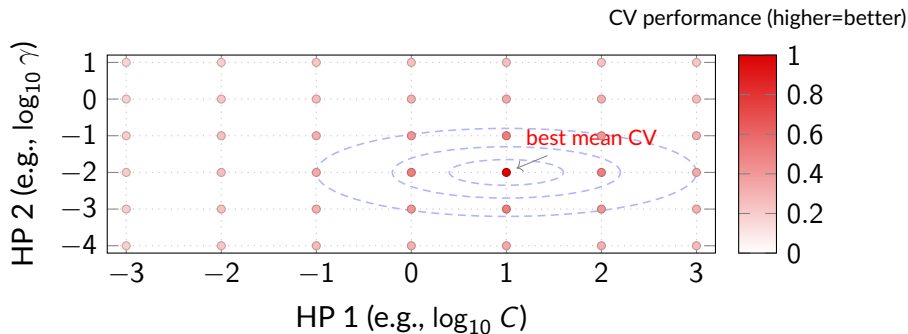
Principles

- **Right scale:** tune on log-scale for multiplicative HPs (e.g., *learning rate* $\in [10^{-5}, 10^{-1}]$ *sampled uniformly in $\log_{10}$*).
- **Ranges from priors:** literature, sanity checks, and pilot runs inform bounds (e.g., *set dropout* $\in [0.0, 0.5]$ *because prior work rarely exceeds 0.5*).
- **Conditional structure:** encode dependencies to avoid invalid configs (e.g., *if kernel=rbf then tune γ ; if kernel=linear then skip γ*).
- **Normalization:** map to $[0, 1]^d$ to help model-based methods (e.g., *optimize $u \in [0, 1]$ and set $lr = 10^{-5+4u}$, $depth = 2 + 18u$*).
- **Compactness vs. coverage:** too wide wastes budget; too narrow risks misspecification (e.g., *start with $lr \in [10^{-4}, 10^{-1}]$; widen only if the best lies at the boundary*).

Grid Search

Idea: Evaluate a Cartesian grid over each HP dimension.

- Strength: *deterministic*, easy to parallelize, reproducible; good for very low d and when interactions are known.
- Weakness: Curse of dimensionality; wastes trials on unimportant dimensions; sensitive to grid choice.



Grid search: evaluate a fixed lattice of HPs with CV, then pick the configuration with the best average validation score.

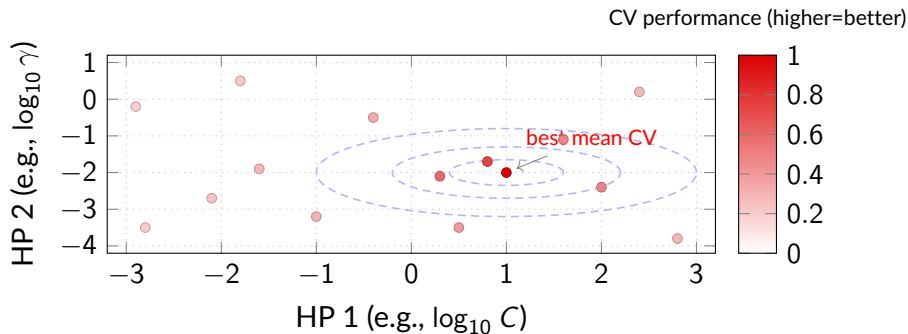
Grid Search Example in scikit-learn

```
1 from sklearn import svm, datasets
2 from sklearn.model_selection import GridSearchCV,
   train_test_split
3 # Load example data
4 X, y = datasets.load_iris(return_X_y=True)
5 X_train, X_test, y_train, y_test = train_test_split(X, y
   , test_size=0.3, random_state=42)
6 # Define model and hyperparameter grid
7 svc = svm.SVC()
8 param_grid = {'C': [0.1, 1, 10, 100], 'gamma': [1, 0.1,
   0.01, 0.001], 'kernel': ['rbf']}
9 # Perform grid search with 5-fold cross-validation
10 grid = GridSearchCV(svc, param_grid, refit=True, cv=5,
   verbose=0)
11 grid.fit(X_train, y_train)
12 print("Best parameters:", grid.best_params_)
13 print("Best CV accuracy:", grid.best_score_)
14 print("Test accuracy:", grid.score(X_test, y_test))
```

Random Search

Idea: Instead of evaluating every grid point, sample random configurations from the hyperparameter space.

- Strength: More efficient in high-dimensional spaces; good for finding near-optimal regions quickly.
- Weakness: Results may vary due to randomness; may miss small but important regions.



Random search: randomly sample hyperparameter combinations; often finds good solutions faster.

Random Search Example in scikit-learn

```
1 from sklearn import svm, datasets
2 from sklearn.model_selection import RandomizedSearchCV,
  train_test_split
3 from scipy.stats import loguniform
4 # Load example data
5 X, y = datasets.load_iris(return_X_y=True)
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
  =0.3, random_state=42, stratify=y)
7 # Define model and hyperparameter distributions
8 svc = svm.SVC()
9 param_distributions = {'C': loguniform(1e-2, 1e2), 'gamma': loguniform
  (1e-4, 1e0), 'kernel': ['rbf']}
10 # Perform random search with 5-fold cross-validation
11 rand = RandomizedSearchCV(estimator=svc, param_distributions=
  param_distributions, n_iter=20, cv=5, random_state=42, n_jobs=-1,
  verbose=0)
12 rand.fit(X_train, y_train)
13 print("Best parameters:", rand.best_params_)
14 print("Best CV accuracy:", rand.best_score_)
15 print("Test accuracy:", rand.score(X_test, y_test))
```

Multi-Fidelity HPO: Successive Halving

Idea: Allocate small budgets to many configs, then *aggressively prune* and *promote* the promising few.

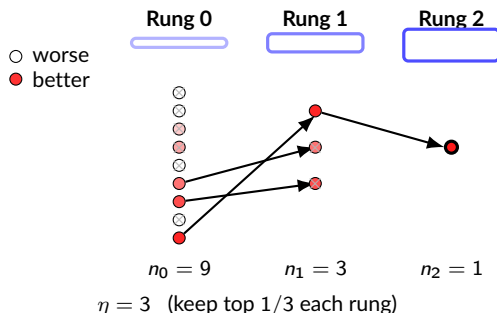
- **Successive Halving (SH):** Evaluate all configs at low resource (epochs/data); keep top $1/\eta$; increase resource; repeat.
- **Hyperband:** Runs multiple SH brackets with different initial budgets to balance exploration vs. exploitation.

When Useful

Training curves are predictive early (learning-curve extrapolation); large search spaces; limited total budget.

Successive Halving

Idea: Start many configs with a small budget; keep the top $1/\eta$ and *increase* the budget per surviving config at each rung.



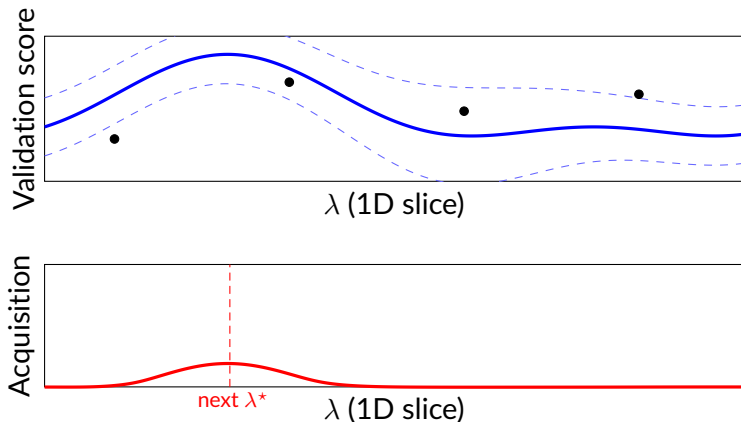
Successive Halving: prune aggressively using small budgets; concentrate resources on the most promising configurations.

Successive Halving Example in scikit-learn

```
1 from sklearn.experimental import enable_halving_search_cv
2 from sklearn.model_selection import HalvingGridSearchCV,
   HalvingRandomSearchCV, train_test_split
3 from sklearn import svm, datasets
4 from scipy.stats import loguniform
5 X, y = datasets.load_iris(return_X_y=True)
6 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
   random_state=42, stratify=y)
7 svc = svm.SVC()
8 param_dist = {'C': loguniform(1e-2, 1e2), 'gamma': loguniform(1e-4, 1
   e0), 'kernel': ['rbf']}
9 shr = HalvingRandomSearchCV(estimator=svc, param_distributions=
   param_dist, factor=3, resource='n_samples', max_resources='auto',
   min_resources=50, random_state=42, n_candidates='exhaust', scoring
   ='accuracy')
10 shr.fit(Xtr, ytr)
11 print("SH best params:", shr.best_params_, "CV acc:", shr.best_score_)
12 print("Test acc:", shr.score(Xte, yte))
```

Bayesian Optimization: Surrogate View

Idea: Build a probabilistic surrogate (i.e., Gaussian process $p(f \mid \mathcal{D})$) of the unknown validation objective $f(\lambda)$ over HP space; pick the next λ by optimizing an acquisition function.



Acquisition Functions: Explore vs. Exploit

Common choices

- **EI** (Expected Improvement): favors high mean and high uncertainty.
- **PI** (Probability of Improvement): maximizes $\Pr(f(\lambda) < f_{\text{best}} - \epsilon)$.
- **UCB**: balances mean and uncertainty via $\mu(\lambda) + \kappa \sigma(\lambda)$ (or maximization variant).

BatchParallel BO

- Penalize neighborhoods of pending points (local penalization) or use Thompson sampling from surrogate.

Tuning the Tuner

Acquisition hyperparameters (e.g., κ , exploration rate) should reflect risk tolerance and budget.

Bayesian Optimization Example (scikit-optimize)

```
1 # pip install scikit-optimize # (once)
2 from skopt import BayesSearchCV
3 from skopt.space import Real, Categorical
4 from sklearn.svm import SVC
5 from sklearn.pipeline import make_pipeline
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.model_selection import StratifiedKFold, train_test_split
8 from sklearn import datasets
9 X, y = datasets.load_iris(return_X_y=True)
10 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
    random_state=42, stratify=y)
11 pipe = make_pipeline(StandardScaler(), SVC())
12 search_spaces = {'svc__C': Real(1e-2, 1e2, prior='log-uniform'), '
    svc__gamma': Real(1e-4, 1e0, prior='log-uniform'), 'svc__kernel':
    Categorical(['rbf'])}
13 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
14 opt = BayesSearchCV(estimator=pipe, search_spaces=search_spaces,
    n_iter=30, cv=cv, scoring='accuracy', n_jobs=-1, random_state=42,
    refit=True, verbose=0)
15 opt.fit(Xtr, ytr)
16 print("Best params:", opt.best_params_)
17 print("Best CV accuracy:", opt.best_score_)
18 print("Test accuracy:", opt.score(Xte, yte))
```

Practice: Bayesian Optimization

Objective: Tune an SVC with Bayesian Optimization (BO).

Fixed setup

- **Dataset:** `sklearn.datasets.breast_cancer` (binary classification).
- **Split:** **70% Train / 30% Test**, stratified, `random_state=42`.
- **Estimator:** `Pipeline(StandardScaler(), SVC(kernel='rbf'))`.
- **Optimizer:** `skopt.BayesSearchCV` with `n_iter=25`,
`cv=StratifiedKFold(5, shuffle=True, random_state=42)`,
`scoring='roc_auc'`, `n_jobs=-1`, `random_state=42`.
- **Search space:** $C \in [10^{-2}, 10^2]$, $\gamma \in [10^{-4}, 10^0]$ (both log-uniform).

What to implement

- 1 Fit `BayesSearchCV` on the Train split; refit the pipeline on full Train with the best hyperparameters.
- 2 Evaluate **once** on Test and report ROC-AUC.
- 3 Print: best hyperparameters, CV mean $\pm$ std (ROC-AUC over 5 folds), and Test ROC-AUC.

Model Ensembles

Why Model Ensembles?

High-Level Idea

Combine multiple *weakly correlated* models to reduce variance and/or bias, yielding a predictor with **better generalization** than any single model.

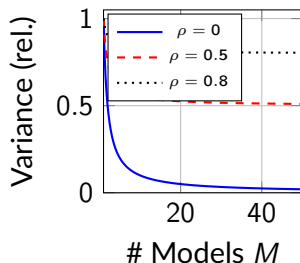
Motivations

- **Variance reduction (averaging):** unstable learners (e.g., trees) benefit from bagging.
- **Bias reduction (sequential correction):** boosting focuses on residual/hard examples.
- **Robustness:** decorrelation via resampling/subsampling/feature sampling improves stability.
- **Practicality:** off-the-shelf performance with modest tuning; strong baselines.

Ensembles and the Bias–Variance Trade-off

For a regression ensemble of M independent base learners $\{f_m\}$ with variance σ_f^2 and common bias b , the variance of the average $\bar{f} = \frac{1}{M} \sum_m f_m$ is roughly $\frac{\sigma_f^2}{M}$ (ignoring correlation).

Takeaway: *Averaging reduces variance.* If base learners are correlated, the reduction is weaker; hence we explicitly **decorrelate** them (bootstrap, feature subsampling).



Key Principle

Variance: use bagging/feature sampling.

Bias: use boosting/stronger base learners.

Noise: cannot be learned away.

Bagging (Bootstrap Aggregating)

Idea: Train M base learners on bootstrap samples (drawing N examples *uniformly at random with replacement* from the training set); aggregate their predictions by averaging (regression) or majority vote (classification).

Algorithm

- 1 For $m = 1, \dots, M$: draw a bootstrap sample $\mathcal{D}^{(m)}$ of size N from $\mathcal{D}$.
- 2 Fit a base learner $f^{(m)}$ on $\mathcal{D}^{(m)}$.
- 3 Output $\hat{f}(x) = \frac{1}{M} \sum_m f^{(m)}(x)$ (or vote).

When it shines

- High-variance, low-bias base learners (e.g., deep decision trees).
- Moderate data sizes; parallelizable training.

Bagging: Bootstrap and Out-of-Bag (OOB) Estimate

- Each bootstrap sample leaves out $\lim_{N \rightarrow \infty} (1 - \frac{1}{N})^N \approx 36.8\%$ of unique examples (OOB set).
- Use OOB examples to estimate generalization without a separate validation set.

Protocol

For each training point i , collect predictions from base learners that did not include i in their bootstrap. Aggregate these to form an **OOB prediction** and compute an **OOB score**.

```
1 from sklearn.ensemble import BaggingClassifier
2 from sklearn.tree import DecisionTreeClassifier
3 clf = BaggingClassifier(
4     base_estimator=DecisionTreeClassifier(max_depth=None),
5     n_estimators=200, oob_score=True, n_jobs=-1, random_state=42)
6 clf.fit(X_train, y_train)
7 print("OOB score:", clf.oob_score_)
```

Random Forests: Decorrelating Trees

Two sources of randomness

- **Bootstrap** samples per tree (as in bagging).
- **Feature subsampling** at each split: choose the best split among a random subset of features of size m_{try} .

Effects

- Reduces inter-tree correlation $\Rightarrow$ stronger variance reduction when averaging.
- Maintains low bias with sufficiently deep trees.

```
1 from sklearn.ensemble import RandomForestClassifier
2 rf = RandomForestClassifier(
3     n_estimators=400,
4     max_depth=None,
5     max_features='sqrt',    # classification default
6     min_samples_leaf=1,
7     n_jobs=-1, random_state=42, oob_score=True)
8 rf.fit(X_train, y_train)
9 print(rf.oob_score_)
```

Key knobs (bias-variance)

- `n_estimators`: variance $\downarrow$ (diminishing returns); parallel.
- `max_depth`, `min_samples_leaf`: deeper $\Rightarrow$ bias $\downarrow$, variance $\uparrow$.
- `max_features` (m_{try}): smaller $\Rightarrow$ decorrelation $\uparrow$ (variance $\downarrow$), but split quality may drop (bias $\uparrow$).
- `bootstrap`, `oob_score`: enable OOB validation.

Tuning checklist (CV/OOB)

- Fix a sufficiently large `n_estimators` (e.g., 300–1000), then adjust `max_features` and `min_samples_leaf`.
- Monitor OOB score as a fast proxy; confirm with CV on Train+Val.
- For noisy labels, increase `min_samples_leaf` or use class-balanced criteria.

Idea: Build an **additive model** $F_M(x) = \sum_{m=1}^M \eta_m f_m(x)$ sequentially, where each new learner f_m focuses on correcting the **residuals** or **hard examples** of the current ensemble.

Characteristics

- Primarily reduces **bias** by fitting more flexible stage-wise corrections.
- Sensitive to noise and outliers without regularization (shrinkage, subsampling, depth limits).
- Strong performance with trees of limited depth (stumps or shallow trees).

AdaBoost (Binary Classification)

Exponential loss minimization with reweighting

- Initialize sample weights $w_i^{(1)} = \frac{1}{N}$.
- For $m = 1, \dots, M$:
 - Fit $f_m(x) \in \{-1, +1\}$ to minimize weighted error
 $\varepsilon_m = \sum_i w_i^{(m)} 1[f_m(x_i) \neq y_i]$.
 - Set learner weight $\alpha_m = \frac{1}{2} \log \frac{1-\varepsilon_m}{\varepsilon_m}$.
 - Update $w_i^{(m+1)} \propto w_i^{(m)} \exp(-\alpha_m y_i f_m(x_i))$ and renormalize.
- Final prediction: $\text{sign}(\sum_m \alpha_m f_m(x))$.

Regularization: use `learning_rate < 1` (shrinkage) and limit tree depth.

```
1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.tree import DecisionTreeClassifier
3 ada = AdaBoostClassifier(
4     base_estimator=DecisionTreeClassifier(max_depth=1), # stumps
5     n_estimators=300, learning_rate=0.05, random_state=42)
6 ada.fit(X_train, y_train)
7 print("Val F1:", cross_val_score(ada, X_train, y_train, cv=5, scoring=
    'f1').mean())
```

Gradient Boosting: Functional Gradient Descent

Stage-wise additive modeling

$$F_0(x) = \arg \min_c \sum_i L(y_i, c), \quad F_m(x) = F_{m-1}(x) + \nu \gamma_m f_m(x)$$

where f_m fits the negative gradient (pseudo-residuals) at F_{m-1} and γ_m is a line-search step (often approximated). **Common losses**

- Regression: squared loss, Huber.
- Classification: logistic loss (binomial deviance).

Regularization

- **Shrinkage** (learning rate ν), **subsampling** (stochastic GBM), **tree depth** (weak learners).

Gradient Boosting in Practice

```
1 from sklearn.ensemble import GradientBoostingClassifier
2 gb = GradientBoostingClassifier(
3     n_estimators=300, learning_rate=0.05,
4     max_depth=3, subsample=0.8, random_state=42)
5 cv = StratifiedKFold(n_splits=5, shuffle=True,
6     random_state=42)
7 scores = cross_val_score(gb, X_train, y_train, cv=cv,
8     scoring='roc_auc')
9 print(f"CV AUC: {scores.mean():.3f}, {scores.std():.3f}")
```

Bias-variance knob: lower learning_rate and increase n_estimators (with early stopping) for smoother fits.

Objective (per iteration)

$$\mathcal{L} = \sum_i L(y_i, F_{m-1}(x_i) + f_m(x_i)) + \Omega(f_m), \quad \Omega(f) = \gamma T + \frac{\lambda}{2} \sum_j w_j^2$$

regularizing the number of leaves T and leaf weights w_j .

Key features

- Second-order (Newton) tree building with efficient histogram-based splits.
- Column (feature) subsampling; row subsampling per tree/level.
- Handling of missing values, sparsity-aware splits.
- Parallel, out-of-core, and GPU support.

XGBoost in Python

```
1 from xgboost import XGBClassifier
2 xgb = XGBClassifier(
3     n_estimators=2000, max_depth=4, learning_rate=0.03,
4     subsample=0.8, colsample_bytree=0.8,
5     reg_lambda=1.0, reg_alpha=0.0,
6     tree_method='hist', n_jobs=-1, random_state=42)
7 xgb.fit(X_train, y_train,
8         eval_set=[(X_val, y_val)], eval_metric='auc',
9         early_stopping_rounds=100, verbose=False)
10 print("Best iter:", xgb.best_iteration)
```

Primary controls

- `eta` (learning rate), `n_estimators`, `max_depth` or `max_leaves`.
- `subsample`, `colsample_bytree/level` for stochastic regularization.
- `lambda` (L2), `alpha` (L1) regularization; `min_child_weight` to avoid overfitting small leaves.

Protocol

- Fix small `eta` (e.g., 0.02–0.1) with early stopping on a validation split.
- Tune `max_depth` (or `leaves`) and `min_child_weight`; then subsampling ratios; finally regularization.

Bagging vs. Boosting: When to Use Which?

Bagging (RF)

- Strength: variance reduction, robust to noise, parallel.
- Works well with many features; minimal tuning.
- Use when base learner is high-variance; when label noise is present.

Boosting (GBM/XGB)

- Strength: bias reduction, state-of-the-art tabular accuracy.
- Requires careful regularization (shrinkage, subsampling).
- Use when subtle interactions exist and noise is moderate.

Operational Guidance

Start with RF as a strong baseline; move to GBM/XGB with early stopping for extra gains; cross-check with nested CV for unbiased reporting.

Practice (To Be Submitted)

Objective: Apply Bayesian Optimization (BO) to tune XGBClassifier (hist-based GBDT) and compare with random search.

Dataset & Metric

- `sklearn.datasets.breast_cancer`; stratified split: **70% Train / 30% Test**.
- Use **ROC-AUC** for model selection and reporting.

Model & Hyperparameters

- Train XGBClassifier with `tree_method='hist'`.
- Tune: η (`learning_rate`), `max_depth`, `min_child_weight`, `subsample`, `colsample_bytree`, `reg_lambda`, `reg_alpha`, `n_estimators` (use early stopping if implemented).

Optimization Protocol

- Implement **BO** with *Optuna* or *scikit-optimize* (*BayesSearchCV*).
- 5-fold **Stratified CV** on the training set; fixed trial budget (e.g., **30** trials).

Baseline

- *RandomizedSearchCV* with the *same* trial budget, CV, and scoring.

Thank You!